Parts of MATLAB

The Interface, Its Components, and How to Use Them

Contents

1. The Overall Layout

When you open MATLAB, you're greeted by several panels arranged around the screen rather than a single blank page. This can feel overwhelming at first, but each panel has one specific job. Once you know what each one does, you'll find yourself using only three or four of them for most of your work: the **Command Window**, the **Editor**, the **Workspace**, and the **Current Folder** browser.

You can drag, resize, undock, or close most of these panels to arrange them however you like — if your layout ever gets messy, go to the **Home** tab on the toolstrip and click **Layout → Default** to reset everything back to normal.

---

2. The Command Window

This is the panel with the >> prompt, usually sitting in the middle of the screen. It's where you type a single line of MATLAB code and see the result immediately after pressing Enter.

```
>> 3 + 4
ans = 7
```

The Command Window is for quick, throwaway calculations and for testing small snippets before putting them into a proper script — think of it as a calculator that also understands variables, matrices, and functions. Anything you type here runs **immediately**, one line at a time; it isn't saved anywhere unless you specifically save your variables or copy the commands elsewhere.

Useful things to know:

- Press the **Up arrow** to bring back your previous command (keep pressing it to scroll further back).
- Typing `clc` clears everything shown in the window, without affecting any variables.
- If a line doesn't end in `;`, MATLAB prints the result of that line right below it, labelled ans = (or the variable name if you assigned it).

---

3. The Editor

The Editor is where you write and save actual MATLAB programs — files ending in `.m`, called **scripts** or **function files**. Unlike the Command Window, code here isn't run line-by-line as you type; you write the whole thing first, save it, and then run it.

To open a new script: **Home tab → New → Script**, or type `edit` in the Command Window. To run the whole file, press the green **Run** button (or F5). You can also **highlight** a section of code and press **F9** to run just that portion — very useful for testing part of a longer script without re-running everything.

The Editor also underlines problems in your code with a red squiggly line (syntax errors) or an orange one (warnings, like an unused variable) — hovering over the underline tells you what's wrong before you even try to run it.

Sections can be marked with %% to break a long script into labelled, independently-runnable chunks:

```
%% Step 1: Load data
data = load('myfile.mat');

%% Step 2: Process data
result = data * 2;
```

Clicking anywhere inside a %% section and pressing **Ctrl+Enter** runs just that section, which is handy for scripts with several distinct stages.

---

4. The Workspace

The Workspace panel (usually top-right) lists every variable currently stored in memory, along with its value, size, and data type. It updates automatically as your code runs.

This is your best tool for checking "did my code actually do what I think it did?" — rather than typing `disp(x)` repeatedly, you can just glance at the Workspace to see the current value of `x`. **Double-clicking** a variable opens the **Variable Editor**, a spreadsheet-like view where you can see and even manually edit the contents of a vector or matrix cell by cell.

Right-clicking a variable gives you options to plot it directly, rename it, or delete it. The workspace is cleared by running `clear` or `clear all` in the Command Window, or by clicking the broom icon at the top of the panel.

---

5. The Current Folder Browser

This panel (usually top-left) shows the files in whichever folder MATLAB is currently "pointed at" — your **current working directory**. This matters because MATLAB can only directly run scripts, and load or save plain files, that live in this folder (or one you've explicitly added to its search path).

The folder path itself is shown and editable in a bar near the top of the window, just above the Editor. If MATLAB says a script or file "cannot be found," the most common cause is that your Current Folder isn't set to where the file actually is — double-click a folder in this browser, or use the cd command, to change it:

```
cd 'C:\Users\YourName\Documents\MATLAB'
```

Double-clicking a `.m` file in this browser opens it in the Editor. Double-clicking a `.mat` file loads its variables straight into the Workspace.

---

6. The Command History

Usually tucked away as a tab or panel near the Command Window, the Command History keeps a running log of every command you've typed in the Command Window, across sessions — even ones from days ago. It's a lifesaver when you know you did something useful earlier but forgot exactly how you wrote it.

You can **double-click** any line in the history to re-run it immediately, or right-click a group of lines and choose **Create Script** to turn a whole block of past commands into a proper, reusable `.m` file.

---

7. The Toolstrip

The Toolstrip is the ribbon of buttons and tabs running along the very top of the window (Home, Plots, Apps, etc.), similar in style to Microsoft Office. This is where most day-to-day actions live without needing to type a command:

- **Home tab:** New script/function, Open, Save, Import Data, Layout options

- **Plots tab:** Appears once you've selected a variable in the Workspace — lets you generate a graph by clicking a chart type instead of writing plotting code
- **Apps tab:** A library of ready-made interactive tools (Curve Fitting, Simulink, etc.)

Beginners often ignore the Plots tab, but it's a genuinely fast way to visualise a vector or matrix without remembering plotting syntax: select the variable in the Workspace, click the Plots tab, and choose a chart type.

---

8. The Figure Window

Whenever you plot something — using `plot`, `bar`, `surf`, and similar functions — MATLAB opens a separate pop-up window called the **Figure window** to display it. This is a separate window entirely, not a panel inside the main MATLAB layout.

```
x = 1:10;
y = x.^2;
plot(x, y);
title('y = x^2');
xlabel('x');
ylabel('y');
```

The toolbar at the top of the Figure window lets you zoom, pan, rotate (for 3D plots), and export the image directly (**File → Save As**, or **File → Export**) as a PNG, PDF, or other format for use in a report. Each new `plot` command by default **replaces** the contents of the current figure — use `figure;` before a plotting command to open a fresh window instead of overwriting the last one, or `hold on` to add new lines to the same figure without erasing what's already there.

---

9. The Path (Search Path)

MATLAB's **path** is the list of folders it's allowed to look inside when you type a command or call a function, beyond just the Current Folder. This mostly runs invisibly in the background — MATLAB's own built-in functions are always on the path — but it becomes important once you start organising a project into multiple folders (e.g. keeping all your custom functions in a separate `functions` folder).

To add a folder permanently: **Home tab → Set Path → Add Folder**. To add one temporarily for just the current session:

```
addpath('C:\Users\YourName\Documents\MyProject\functions')
```

If MATLAB says a function you wrote "is not recognised," and you're sure you saved the file, this is usually a path problem rather than a typo — the file exists, but MATLAB isn't looking in that folder.

---

10. Live Scripts (.mlx files)

A **Live Script** is a more modern alternative to a plain `.m` script, saved with a `.mlx` extension. It looks like a notebook: your code, its output, and any plots all appear together in one continuous, formatted document, rather than code in the Editor and results in a separate Figure window or Command Window.

To create one: **Home tab → New → Live Script**. You can mix in formatted text, headings, equations, and images alongside your code, which makes Live Scripts a good choice for lab reports or assignments where you need to present both your working and your explanation together, since the whole thing can be exported straight to PDF (**Save As → PDF**).

---

11. The Help / Documentation

If you're not sure what a function does or what inputs it expects, you rarely need to search online first — MATLAB's documentation is built in and usually more reliable.

```
help plot        % quick text explanation in the Command Window
doc plot         % opens the full documentation page, with examples
```

`help` gives you a fast, no-frills summary right where you're working; `doc` opens a nicer formatted page in a separate documentation window, complete with worked examples you can often copy and adapt directly. There's also a search bar in the top-right corner of the whole MATLAB window for browsing documentation by topic instead of by exact function name.

Summary — Quick Reference

Part	Where it is	What it's for
Command Window	Centre panel, >> prompt	Run one line of code at a time; quick tests and calculations
Editor	Opens when you create/open a `.m` file	Write and save full scripts and functions; run with F5
Workspace	Top-right panel	See every variable currently in memory, its size and type
Current Folder	Top-left panel	Shows/changes which folder MATLAB is working in
Command History	Tab near Command Window	Log of every past command; double-click to re-run
Toolstrip	Ribbon along the top	Buttons for New, Save, Plots, Apps, Layout, etc.
Figure window	Pops up on `plot` etc.	Displays graphs; zoom, export, or save as image/PDF

Part	Where it is	What it’s for
Path	Background setting	List of folders MATLAB searches for functions/scripts
Live Script (.mlx)	New → Live Script	Notebook-style file combining code, text, and output
Help / doc	`help`, `doc`, or search bar	Built-in documentation and examples for any function

Practical tips for beginners:

1. Use the **Command Window** to test a quick idea; use the **Editor** for anything you want to save and reuse.
2. If MATLAB can’t find your script, check the **Current Folder** first — it’s almost always the cause.
3. Keep an eye on the **Workspace** panel instead of sprinkling `disp()` everywhere to check your variables.
4. Use `figure;` before plotting if you don’t want to overwrite your last graph.
5. When stuck on a function, type `doc functionname` before searching the internet — MATLAB’s own examples are usually enough.